

BÀI TẬP VỀ NHÀ – LỜI GIẢI CHI TIẾT PHẦN A & B

(Ôn tập Bài 17 → 21)

PHẦN A — Lời giải Lý thuyết

1. Suốt Bài 1–16 bạn tự viết CSS. Bài 17 bạn chuyển sang Material. Nêu ba thứ bạn được và hai thứ bạn mất khi dùng thư viện UI.

► ĐƯỢC:

1. Tốc độ phát triển: Không phải code lại từ đầu các component phức tạp (Datepicker, Dialog, Select...).
2. Chuẩn hóa UI/UX: Giao diện đẹp, đồng bộ và tuân thủ chặt chẽ các quy chuẩn về Khả năng tiếp cận (Accessibility - a11y).
3. Độ tin cậy cao: Thư viện đã được cộng đồng test kỹ ở nhiều trình duyệt.

► MẤT:

1. Sự tự do: Rất gò bó, khó tùy biến (custom) lại giao diện nếu designer vẽ một thứ đi ngược lại với triết lý của thư viện.
2. Hiệu năng / Bundle size: Ứng dụng nặng hơn vì chứa lượng lớn code CSS/JS của thư viện.

2. Ng add khác npm install ở điểm nào? Vì sao việc "tự cấu hình hệ" lại có ích và có rủi ro?

- Khác biệt: 'npm install' chỉ tải source code về thư mục node_modules. Còn 'ng add' vừa tải code vừa chạy script tự động sửa file trong dự án (như angular.json, styles.css).
- Có ích: Tiết kiệm thời gian, tránh lỗi do tự cấu hình bằng tay sai hoặc thiếu bước.
- Rủi ro: Có thể ghi đè/phá hỏng cấu hình hiện tại nếu không kiểm soát. Luôn cần commit Git trước khi chạy ng add.

3. Vì sao nên import từng module cụ thể (@angular/material/button) thay vì cả gói? Điều này liên quan gì tới thời gian tải trang của người dùng?

- Việc import từng module giúp Angular áp dụng cơ chế Tree-shaking khi build app, chỉ lấy những đoạn code thực sự dùng. Nếu import cả gói lớn, toàn bộ code thư viện (dù không dùng) cũng bị đóng gói gửi đi, làm phình to file JS, kéo dài thời gian tải trang và gây chậm thiết bị người dùng.

4. Ở Bài 17 bạn từng đặt <mat-icon> vào một khe mà thư viện dành cho chữ, và icon biến mất mà không có lỗi nào báo ra. Rút ra bài học gì về cách làm việc với thư viện UI nói chung? Khi gặp tình huống "trông sai nhưng không báo lỗi", bạn tra ở đâu?

- Bài học: Làm việc với thư viện UI phải tuân thủ nghiêm ngặt cấu trúc HTML (khe/slot) mà họ quy định. Trình biên dịch sẽ không bắt được lỗi logic giao diện này.
- Cách tra cứu: Tra trực tiếp tại Tài liệu chính thức (Docs) của thư viện phần API và cấu trúc mẫu, kết hợp dùng F12 (DevTools) trên trình duyệt để kiểm tra cấu trúc DOM.

5. Một nút có chữ hiển thị "Thêm món mới" mà lại gắn aria-label="Add item". Người dùng trình đọc màn hình nghe thấy gì? Rút ra quy tắc khi nào nên và khi nào không nên dùng aria-label.

- Nghe thấy: "Add item" (bằng tiếng Anh). Thuộc tính aria-label sẽ ghi đè hoàn toàn nội dung chữ bên trong nút.

- Quy tắc: Chỉ dùng aria-label cho những nút CHỈ CÓ BIỂU TƯỢNG (icon) mà không có chữ hiển thị. Nếu nút đã có chữ rõ ràng thì tuyệt đối không dùng aria-label để tránh gây nhiễu thông tin.

6. Vì sao bọc một biểu tượng chỉ để trang trí trong thẻ <button> là sai? Nêu hai hậu quả cụ thể với người dùng.

- Hậu quả 1: Trình đọc màn hình sẽ đọc đây là một "nút bấm", người khiếm thị sẽ click thử và hoang mang vì không có chức năng gì xảy ra.
- Hậu quả 2: Người dùng phím Tab sẽ bị khựng lại ở biểu tượng này (vì thẻ button nhận focus mặc định), làm gián đoạn luồng điều hướng của trang web.

7. Một dòng trong danh sách bấm chuột thì chuyển trang được, nhưng nhấn Tab thì con trỏ nhảy qua. Nguyên nhân gốc là gì? (Gợi ý: nghĩ về loại thẻ HTML chứ không phải thư viện.)

- Nguyên nhân gốc: Do sử dụng sai thẻ HTML. Đang dùng các thẻ hiển thị cấu trúc (như <div>,) và gắn sự kiện (click). Trình duyệt mặc định không cho phép Tab (focus) vào các thẻ này. Dòng điều hướng bắt buộc phải dùng thẻ <a> (có href/routerLink) hoặc thẻ <button>.

8. Ô nhập liệu cần được "nối" với nhãn của nó. Nêu hai cách làm điều đó, và cho biết vì sao để thư viện lo hộ thì an toàn hơn tự nối tay.

- Cách 1 (Thủ công): Dùng thuộc tính 'for' của <label> trỏ vào 'id' của <input>.
- Cách 2 (Thư viện UI): Bọc tất cả vào một component cha như <mat-form-field>.
- Vì sao an toàn hơn: Tự nối tay rất dễ đặt trùng 'id' trên cùng một màn hình khiến nhãn nối sai mục tiêu. Thư viện tự sinh ra các 'id' ảo duy nhất, đảm bảo không bao giờ nối nhầm.

9. Reactive Forms giữ bản sao dữ liệu bên trong form. Signal Forms thì bọc quanh dữ liệu gốc của bạn. Nêu hai hệ quả thực tế của khác biệt này khi bạn viết hàm save().

- Hệ quả 1: Lấy dữ liệu cực dễ. Không cần dùng .value hay .getRawValue(), chỉ cần đọc thẳng giá trị hiện tại từ signal gốc (VD: this.drinkModel()).
- Hệ quả 2: Phải reset hai bước. Hàm reset() của form chỉ xóa các cờ trạng thái UI (touched, errors). Để xóa nội dung ô nhập, bạn phải tự cập nhật lại signal gốc (this.drinkModel.set(...)).

10. submit() tự động làm ba việc trước khi chạy hành động của bạn. Kể ra, và giải thích vì sao để thư viện lo ba việc đó tốt hơn tự viết tay.

- 1. Chặn gửi trùng: Trả false ngay nếu form đang được gửi, ngăn lỗi double-click gây spam server.
- 2. markAllAsTouched(): Tự động kích hoạt hiển thị toàn bộ lỗi nếu form sai, cảnh báo người dùng ngay lập tức.
- 3. Chặn khi không hợp lệ: Không chạy hành động lưu nếu form vi phạm validate. Giúp code gọn hơn, không cần phải tự viết 'if (invalid) return' thủ công.

11. Gọi reset() mà ô input vẫn còn chữ. Vì sao lại thiết kế như vậy? (Gợi ý: liên hệ với câu 9 – ai là chủ sở hữu dữ liệu?)

- Vì Signal Forms tuân thủ triết lý "Ai nắm dữ liệu người đó quyết định". Hàm reset() của Form chỉ có quyền dọn dẹp các cờ trạng thái giao diện (touched, dirty, thông báo lỗi). Nội dung chữ là do Signal Model sở hữu, muốn xóa chữ thì bạn phải tác động vào biến gốc bằng model.set().

12. Vì sao chỉ nên hiện thông báo lỗi khi ô vừa sai vừa đã được người dùng chạm vào? Mô tả trải nghiệm tệ nếu bỏ điều kiện thứ hai.

► Trải nghiệm tệ: Nếu bỏ điều kiện "đã chạm", ngay khi vừa mở form thêm mới (các ô trống vi phạm luật bắt buộc), toàn bộ màn hình sẽ đỏ rực báo lỗi. Điều này gây ấn tượng xấu và gây áp lực lên người dùng khi họ chưa kịp làm gì. Cần đợi họ gõ xong và chuyển sang ô khác (touched) mới đánh giá.

13. <mat-error> của Material vốn sinh ra cho Reactive Forms, vậy mà chạy được với Signal Forms. Cơ chế nào cho phép hai thứ khác thế hệ nói chuyện được với nhau? Kỹ thuật này có tên gọi chung là gì trong lập trình?

► Cơ chế: Signal Forms dùng directive [formField] làm cầu nối, nó chuyển dịch trạng thái lỗi của Signal Form sang định dạng mà các component Material cũ hiểu được.
► Tên kỹ thuật: Mẫu thiết kế Cầu nối (Adapter Pattern) hoặc Interface / Dependency Injection chung.

14. Trong @for, vì sao track bằng chính đối tượng lại gây dựng lại DOM, còn track bằng một chuỗi ổn định thì không? Liên hệ với bài học track ở Bài 11.

► Khi track bằng đối tượng, mỗi khi mảng dữ liệu load lại từ server, các object mới được tạo ra trên RAM. Angular so sánh địa chỉ bộ nhớ thấy khác nhau nên đập bỏ thẻ HTML cũ vẽ thẻ mới (gây giật lag). Track bằng một chuỗi ổn định (như id hoặc error.kind) giúp Angular nhận diện đúng phần tử cũ và tái sử dụng thẻ DOM HTML.

15. So sánh color: #93401f và color: var(--mat-sys-primary). Nêu ba lợi ích của cách thứ hai.

► Lợi ích 1: Bảo trì tập trung. Đổi giá trị token một lần trong theme, cả dự án đổi màu theo.
► Lợi ích 2: Chế độ Tối (Dark mode) miễn phí. Token sẽ tự động trả về mã màu phù hợp tùy theo đang ở theme Sáng hay Tối.
► Lợi ích 3: Code dễ đọc (Semantic). Tên token nói lên ý đồ thiết kế (đây là màu chính) thay vì phải đoán ý nghĩa của một chuỗi mã hex vô hồn.

16. Material 3 phân tầng giao diện bằng độ đậm nhạt của nền thay vì bóng đổ như Material 2. Nêu hai lý do khiến Google đổi hướng.

► Lý do 1: Hỗ trợ Chế độ Tối (Dark mode). Bóng đổ đen trên nền đen là thứ gần như vô hình, không thể hiện được sự phân tầng. Dùng nền đậm/nhạt thấy rõ ở cả 2 chế độ.
► Lý do 2: Hiệu năng và Dịu mắt. Vẽ bóng đổ (shadow) tốn nhiều tài nguyên xử lý hơn so với tô nền tĩnh, đồng thời quá nhiều bóng đổ khiến thiết kế trở nên rối rắm, "ồn ào".

17. Tiền tố on- trong --mat-sys-on-surface nghĩa là gì? Vì sao dùng đúng cặp nền/chữ thì bạn không phải tự kiểm tra độ tương phản WCAG?

► Ý nghĩa: Tiền tố 'on-' báo hiệu đây là màu (thường dùng cho chữ/icon) được dùng để ĐẶT LÊN TRÊN màu nền tương ứng (surface).
► Vì sao không cần kiểm tra: Vì Google đã tính toán sẵn thuật toán để cặp màu này luôn tương phản với nhau ở mức tối thiểu 4.5:1, đạt chuẩn khả năng tiếp cận WCAG cho người khiếm thị.

18. Vì sao nên dùng padding: 16px thay vì padding: 13px? Nguyên tắc này có tên gì, và bạn có gặp lại nó ở Tailwind không?

► Nguyên nhân: Não bộ con người phản ứng tốt với các tỷ lệ thống nhất. Việc tuân theo nhịp độ chia hết cho 8 giúp bố cục ngay ngắn và hài hòa.

- ▶ Tên nguyên tắc: Hệ thống lưới 8dp (8-point grid system).
- ▶ Trong Tailwind: Có. Tailwind cũng dùng thang khoảng cách là bội số của 4/8 (ví dụ class p-4 tức là padding 16px).

PHẦN B — Tìm lỗi trong code

Dưới đây là lời giải chi tiết (chỉ ra lỗi, giải thích và viết lại code đúng) cho 12 đoạn mã trong bài.

Lỗi số 1

```
import { MatButtonModule } from '@angular/material/button';
@Component({
  selector: 'app-drink-list',
  imports: [],
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {}
<button matButton="filled">Lưu</button>
```

- ▶ Lỗi ở đâu: Mảng imports: [] đang trống rỗng.
- ▶ Tại sao: Thẻ <button matButton> trong HTML cần có MatButtonModule để hoạt động. Nếu quên khai báo trong imports, Angular sẽ bỏ qua directive matButton và render thành một nút HTML xấu xí mặc định.
- ▶ Sửa lại:
imports: [MatButtonModule],

Lỗi số 2

```
<button matButton="filled" aria-label="Example icon button with a plus icon">
  <mat-icon>add</mat-icon>
  Thêm món mới
</button>
```

- ▶ Lỗi ở đâu: Thừa thuộc tính aria-label.
- ▶ Tại sao: Nút này đã chứa sẵn đoạn chữ "Thêm món mới" hiển thị rất rõ ràng trên màn hình. Thuộc tính aria-label sẽ ghi đè đoạn chữ này khi trình đọc màn hình quét qua, gây nhiễu thông tin (đặc biệt khi aria-label lại ghi bằng tiếng Anh).
- ▶ Sửa lại:
<button matButton="filled">
 <mat-icon>add</mat-icon>
 Thêm món mới
</button>

Lỗi số 3

```
<mat-toolbar>
  <button matIconButton>
    <mat-icon>local_cafe</mat-icon>
  </button>
```

```
<span>Quầy Trà Sữa</span>
</mat-toolbar>
```

- Lỗi ở đâu: Thẻ `<button matIconButton>` thiếu thuộc tính `aria-label`.
- Tại sao: Nút này CHỈ chứa biểu tượng (icon), không hề có chữ. Đối với người dùng khiếm thị xài trình đọc màn hình, nút này là "vô hình" hoặc sẽ đọc thành "unlabelled button".
- Sửa lại:
`<button matIconButton aria-label="Menu Quầy Trà Sữa">`

Lỗi số 4

```
<mat-nav-list>
  @for (drink of drinks(); track drink.id) {
    <mat-list-item [routerLink]="['/drinks', drink.id]">
      <a matListItemTitle>{{ drink.name }}</a>
    </mat-list-item>
  }
</mat-nav-list>
```

- Lỗi ở đâu: Cấu trúc HTML bọc thẻ sai. Thẻ `<mat-list-item>` đang bọc thẻ `<a>`.
- Tại sao: Trong Material, nếu muốn một dòng trong danh sách có thể điều hướng bằng Tab (tabbable) và hoạt động như một liên kết, thì chính thẻ `<a>` phải đóng vai trò là list-item, không phải nằm bên trong.
- Sửa lại:
`<a mat-list-item [routerLink]="['/drinks', drink.id]">`
`<span matListItemTitle>{{ drink.name }}</span>`
`</a>`

Lỗi số 5

```
import { form, Field } from '@angular/forms/signals';
@Component({
  selector: 'app-add-drink',
  imports: [Field],
  templateUrl: './add-drink.html',
  styleUrls: ['./add-drink.css'],
})
export class AddDrink {}
```

- Lỗi ở đâu: Import nhầm 'Field' thay vì 'FormField'.
- Tại sao: 'Field' chỉ là một Kiểu dữ liệu (Type/Interface) trong TypeScript, không thể đưa vào mảng imports. Thứ cần đưa vào mảng imports để hoạt động trong HTML là Directive 'FormField'.
- Sửa lại:
`import { form, FormField } from '@angular/forms/signals';`
`@Component({`
 `imports: [FormField],`
 `...`
`})`

Lỗi số 6

```
import { form, FormField } from '@angular/forms';
```

- Lỗi ở đâu: Sai đường dẫn thư viện (path).
 - Tại sao: Signal Forms hiện đang là tính năng thử nghiệm của Angular 21, nó nằm trong thư mục `con/signals`. Việc import từ thư mục gốc `@angular/forms` sẽ gây lỗi không tìm thấy.
 - Sửa lại:
- ```
import { form, FormField } from '@angular/forms/signals';
```

## Lỗi số 7

```
<input matInput [field]="drinkForm.name" />
```

- Lỗi ở đâu: Dùng sai tên thuộc tính `[field]`.
  - Tại sao: Theo API của Signal Forms, để nối ô nhập liệu với biến form tương ứng, ta phải dùng cú pháp directive là `[formField]`.
  - Sửa lại:
- ```
<input matInput [formField]="drinkForm.name" />
```

Lỗi số 8

```
<form [formRoot]="drinkForm" (submit)="save($event)">
  <button type="submit">Lưu</button>
</form>

protected async save(event: Event): Promise<void> {
  event.preventDefault();
  await submit(this.drinkForm, async () => {
    this.drinkService.addDrink(/* ... */);
  });
}
```

- Lỗi ở đâu: Vừa dùng thuộc tính `[formRoot]` trên thẻ HTML, vừa tự gọi hàm `submit()` trong file TypeScript.
 - Tại sao: Cả hai công cụ này đều thực hiện chức năng "gửi form" của thư viện. Sử dụng chung sẽ tạo ra xung đột và sinh ra lỗi. Khi đã tự kiểm soát bằng hàm `submit()` thì phải gỡ `[formRoot]` ra.
 - Sửa lại (HTML):
- ```
<form novalidate (submit)="save($event)">
```

## Lỗi số 9

```
const ok = await submit(this.drinkForm, async () => {
 this.drinkService.addDrink(/* ... */);
});
this.drinkForm().reset();
this.drinkModel.set({ name: '', description: '', giaCoBan: 0 });
```

- Lỗi ở đâu: Việc xóa dọn form bị đặt TRÊN ra ngoài vòng kiểm tra thành công (ok).
- Tại sao: Biến 'ok' trả về true nếu form hợp lệ và hành động chạy thành công, trả về false nếu form báo lỗi

vi phạm. Đoạn code sai sẽ khiến việc xóa rỗng form bị chạy LIÊN TỤC kể cả khi người dùng gõ sai và đang cần sửa lại.

► Sửa lại:

```
if (ok) {
 this.drinkForm().reset();
 this.drinkModel.set({ name: "", description: "", giaCoBan: 0 });
}
```

### Lỗi số 10

```
protected async save(event: Event): Promise<void> {
 event.preventDefault();
 await submit(this.drinkForm);
}
```

► Lỗi ở đâu: Hàm submit() bị gọi thiếu tham số bắt buộc.

► Tại sao: Hàm submit() yêu cầu tham số thứ hai là một hàm callback (hành động gửi dữ liệu). Nếu thiếu, Angular sẽ ném lỗi NG1915 văng app.

► Sửa lại:

```
await submit(this.drinkForm, async () => {
 // Code hành động gửi form ở đây
});
```

### Lỗi số 11

```
@for (error of drinkForm.name().errors(); track error) {
 <mat-error>{{ error.message }}</mat-error>
}
```

► Lỗi ở đâu: Thuộc tính track trong @for đang trỏ thẳng vào biến đối tượng 'error'.

► Tại sao: Việc track nguyên một đối tượng JavaScript sẽ khiến Angular nhầm lẫn mỗi khi trạng thái lỗi được cập nhật lại, hệ quả là toàn bộ phần tử HTML liên quan bị đập đi xây lại (gây giảm hiệu năng). Cần track một chuỗi/số định danh nguyên thủy.

► Sửa lại:

```
@for (error of drinkForm.name().errors(); track error.kind) {
```

### Lỗi số 12

```
protected readonly drinkForm = form(this.drinkModel, (path) => {
 required(path.name, 'Tên món là bắt buộc.');
```

► Lỗi ở đâu: Sai cú pháp truyền thông báo lỗi cho hàm luật (validator).

► Tại sao: API của Signal Forms yêu cầu cấu hình tùy chọn (như thông báo lỗi tùy chỉnh) phải được đặt bên trong một object { message: ... }.

► Sửa lại:

```
required(path.name, { message: 'Tên món là bắt buộc.' });
```